

Manipulando texto com Python

Trabalhando com Texto em Python I — Unidade 1

Do texto simples às expressões regulares e ao processamento de múltiplos arquivos.

Adaptado de *Applied Language Technology* (Universidade de Helsinque) — applied-language-technology.mooc.fi

Objetivos de aprendizagem

Ao final desta unidade, você deverá ser capaz de:

- compreender a diferença entre texto formatado (*rich text*), estruturado (*structured text*) e simples (*plain text*);
- entender o conceito de codificação de caracteres (*text encoding*);
- carregar arquivos de texto simples no Python e manipular o seu conteúdo;
- definir padrões flexíveis de busca com expressões regulares;
- processar vários arquivos de uma só vez e salvar os resultados.

1. Computadores e texto

Os computadores podem armazenar e representar texto em formatos diferentes. Conhecer a distinção entre esses tipos é **fundamental** para processá-los programaticamente, porque cada formato traz consigo informações (ou ruídos) distintos.

1.1 O que é texto formatado (*rich text*)?

Processadores de texto, como o Microsoft Word, produzem **texto formatado** (*rich text*): texto cuja aparência foi estilizada de alguma maneira específica. É possível definir estilos visuais para os elementos do documento — títulos com fontes próprias, trechos em itálico ou negrito para dar ênfase, além de imagens e tabelas. Esse é o formato padrão dos editores modernos do tipo *WYSIWYG* (*what you see is what you get*).

1.2 O que é texto simples (*plain text*)?

Diferentemente do texto formatado, o **texto simples** (*plain text*) não contém nenhuma informação sobre a aparência visual: ele é feito **apenas de caracteres** — letras, números, sinais de pontuação, espaços e quebras de linha. A definição é flexível, mas, em geral, refere-se a texto sem qualquer formatação ou estilo.

1.3 O que é texto estruturado (*structured text*)?

O **texto estruturado** pode ser entendido como um caso especial de texto simples que inclui sequências de caracteres usadas para **formatar o texto para exibição**. São exemplos os textos descritos por linguagens de marcação como XML, Markdown ou HTML. Abaixo, uma frase em texto simples envolvida

por marcadores (*tags*) HTML de parágrafo `<p>`. A marca de abertura `<p>` e a de fechamento `</p>` informam ao computador que o conteúdo entre elas forma um parágrafo:

Python

```
<p>This is an example sentence.</p>
```

1.4 Por que isso importa?

Ao reunir textos para formar um **corpus**, é provável que parte deles tenha origem em formato formatado ou estruturado:

- Documentos impressos digitalizados por **reconhecimento óptico de caracteres** (*OCR*) e convertidos para texto simples tendem a acumular **erros**. Esse OCR “sujo” pode afetar a análise textual (Hill & Hengchen, 2019).
- Documentos raspados de fóruns ou sites costumam trazer vestígios de marcadores de marcação, arrastados para o texto simples durante a conversão.

O texto simples é, de longe, o formato **mais interoperável**, por ser fácil de ler para os computadores — por isso as linguagens de programação trabalham com ele.

Em resumo

Ao trabalhar com texto simples, muitas vezes será preciso lidar com os vestígios deixados pela conversão a partir de texto formatado ou estruturado.

2. Codificação de texto (*text encoding*)

Para serem lidos pelos computadores, os textos simples precisam ser **codificados**. A **codificação de caracteres** mapeia cada caractere (letras, números, pontuação, espaços...) para uma representação numérica compreendida pela máquina. O ideal seria não termos de lidar com isso — mas, na prática, precisamos, porque existem **vários sistemas** de codificação e eles **não são compatíveis entre si**.

2.1 ASCII

O **ASCII** (*American Standard Code for Information Interchange*) é um sistema pioneiro que serviu de base para muitos sistemas modernos. Ainda é bastante usado, mas é **muito limitado**: se o seu idioma inclui caracteres como `ä`, `ö` — ou os nossos `á`, `ç`, `ã` —, o ASCII não dá conta.

2.2 Unicode

O **Unicode** é um padrão para codificar a maioria dos sistemas de escrita do mundo, cobrindo cerca de **140 mil caracteres** entre escritas modernas e históricas, símbolos e emojis. Por exemplo, o emoji de fatia de pizza tem o código `U+1F355`, e o espaço em branco corresponde a `U+0020`. O Unicode pode ser implementado por diferentes codificações, como o **UTF-8**.

Ponto-chave

O UTF-8 é **retrocompatível com o ASCII**: as codificações do ASCII são um subconjunto do UTF-8. Assim, um arquivo em ASCII pode ser decodificado como UTF-8 — mas o **contrário não vale**.

3. Carregando arquivos de texto simples no Python

Arquivos de texto simples são carregados com a função `open()`. O primeiro argumento deve ser uma *string* com o **caminho** do arquivo. Aqui, o caminho aponta para `NYT_1991-01-16-A15.txt`, no diretório `data` — diretório e nome separados por uma barra `/`:

Python

```
open(file='data/NYT_1991-01-16-A15.txt', mode='r', encoding='utf-8')
```

- **encoding**: o Python 3 assume UTF-8 por padrão, mas deixamos explícito passando `utf-8`.
- **mode**: a *string* `r` (de *read*) indica que queremos **apenas ler**.
- Usamos `open()` junto do `with`, que garante o **fechamento** do arquivo após o bloco indentado — evitando consumo desnecessário de memória.

Python

```
# Abre um arquivo e o atribui à variável 'file'
with open(file='data/NYT_1991-01-16-A15.txt', mode='r', encoding='utf-8') as file:

    # A instrução 'with' deve ser seguida por um bloco de código indentado.
    # Aqui chamamos o método read() para ler o conteúdo do arquivo e
    # atribuímos o resultado à variável 'text'.
    text = file.read()
```

A palavra `as` instrui o Python a atribuir o retorno de `open()` à variável `file`. Ao chamá-la, obtemos um objeto `TextIOWrapper`:

Python

```
# Chama a variável para examinar o objeto
file
```

Saída

```
<_io.TextIOWrapper name='data/NYT_1991-01-16-A15.txt' mode='r' encoding='utf-8'>
```

Se tentarmos chamar `read()` **fora** do bloco `with`, o Python levanta um erro, porque o arquivo já foi fechado:

Python

```
# Tenta usar o método read() para ler o conteúdo do arquivo
file.read()
```

Saída

```
ValueError: I/O operation on closed file.
```

Esse comportamento é o esperado — importante sobretudo ao trabalhar com **milhares de arquivos**. Vejamos o conteúdo lido, guardado em `text`. Como é longo, pegamos uma **fatia** dos primeiros 500 caracteres com `[:500]`. Adicionar colchetes após o nome da variável permite acessar partes do objeto; `text[1]` recuperaria o caractere na posição 1, e o `:` antes do número recupera todos os caracteres até a posição 500.

Python

```
# Recupera os primeiros 500 caracteres da variável 'text'
text[:500]
```

Saída

```
'U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. Is Ordered to Find Iraqis
Whose Visas Have Expired\nBy JAMES BARRON\nNew York Times (1923-Current file);
Jan 16, 1991; ... the Justice Department'
```

Há sequências estranhas: uma marca BOM (`U+FEFF`) no início e vários `\n`. A primeira é a “assinatura” de que o arquivo é UTF-8; as demais indicam **quebra de linha**. Isso fica evidente ao usar `print()`:

Python

```
# Imprime os primeiros 1000 caracteres da variável 'text'
print(text[:1000])
```

Saída

```
U.S. TAKING STEPS TO CURB TERRORISM: F.B.I. Is Ordered to Find Iraqis Whose Visas
Have Expired
By JAMES BARRON
New York Times (1923-Current file); Jan 16, 1991;
...
The Federal Bureau of Investigation has been ordered to track down as many as
3,000 Iraqis in this country whose visas have expired, the Justice Department
said yesterday.
```

O Python interpreta o `\n` e insere a quebra de linha. As primeiras linhas contêm **metadados** (nome, autor, fonte), que antecedem o corpo do texto.

4. Manipulando texto

Como `text` é um objeto *string*, podemos usar todos os métodos de manipulação de *strings* do Python.

4.1 Substituindo com `replace()`

Vamos trocar todas as quebras de linha `"\n"` por *strings* vazias `""`:

Python

```
# Substitui as quebras de linha \n por strings vazias e atribui o resultado
# à variável 'processed_text'
processed_text = text.replace('\n', '')

# Imprime os primeiros 1000 caracteres da variável 'processed_text'
print(processed_text[:1000])
```

Agora o texto ficou “grudado”, mas ainda dá para identificar o início de cada parágrafo, marcado por **três espaços**.

Atenção

Remover as quebras de linha também fundiu os metadados em um único parágrafo. Sempre fique atento aos **efeitos indesejados** de substituições e transformações!

4.2 Dividindo com `split()`

Sabendo que os metadados estão separados do corpo por três espaços, usamos `split()` para **dividir a string em uma lista**, com os três espaços como separador:

Python

```
# Usa o método split() com três espaços como separador. Atribui o
# resultado à variável 'processed_text'.
processed_text = processed_text.split(sep='   ')

# Imprime o resultado da variável 'processed_text'
print(processed_text)
```

O método retorna uma **lista** de *strings*:

Python

```
# Verifica o tipo do objeto na variável 'processed_text'
type(processed_text)
```

Saída

list

Os metadados ficaram no **primeiro item** (índice 0 — o Python conta do zero):

Python

```
# Recupera o objeto string no índice 0 da lista 'processed_text'
processed_text[0]
```

4.3 Removendo itens com `pop()`

Para remover os metadados, usamos `pop()`, que espera o índice do item a remover:

Python

```
# Chama o método pop() da lista 'processed_text' com o
# índice do item a ser removido.
processed_text.pop(0)
```

Não atribuímos o resultado a uma variável porque as listas são **mutáveis**: `pop()` altera a lista *in place*. Conferindo os três primeiros itens, o metadado já não aparece:

Python

```
# Recupera os três primeiros itens da lista 'processed_text'
processed_text[:3]
```

4.4 Reunindo com `join()`

Para voltar de lista para *string*, usamos `join()`, que espera um **iterável**. O ponto que confunde: o `join()` é chamado **sobre a string que serve de “cola”**. Aqui, a cola é a sequência original de parágrafo (`'\n '` — quebra de linha e três espaços):

Python

```
# Usa o método join() para unir os itens da lista 'processed_text'
# usando a string '\n    ' -- uma quebra de linha e três espaços. Guarda o
# resultado na variável de mesmo nome.
processed_text = '\n    '.join(processed_text)

# Verifica o resultado imprimindo os primeiros 1000 caracteres da string
# resultante em 'processed_text'
print(processed_text[:1000])
```

O `join()` devolve uma *string* com as quebras de parágrafo originais!

4.5 Um *pipeline* de substituições com `for`

O OCR deixou uma **mistura de aspas**. Para padronizar tudo em `"` sem repetir `replace()` à mão, combinamos **listas** e **tuplas**. Cada tupla guarda o caractere a substituir e o seu substituto:

Python

```
# Define uma lista com quatro tuplas, cada uma com duas strings: o caractere
# a ser substituído e o seu substituto.
pipeline = [('"', ' '), ('"', ' '), ('`', ' '), ('`', ' ')]
```

Isso mostra como estruturas de dados são **aninhadas**: a lista contém tuplas, e as tuplas contêm *strings*. Agora percorremos a lista com um `for` (a próxima linha deve ser **indentada**):

```
# Percorre as tuplas da lista 'pipeline'. Cada tupla tem dois valores, que
# atribuímos as variáveis 'old' e 'new' automaticamente!
for old, new in pipeline:

    # Usa o método replace() para substituir a string da variável 'old'
    # pela string da variável 'new'
    processed_text = processed_text.replace(old, new)
```

Python

Saída

Python

7

Python

```
# Define o caminho do arquivo e o abre para leitura (r) com codificacao utf-8
with open(file='data/WP_1990-08-10-25A.txt', mode='r', encoding='utf-8') as file:

    # Le o conteudo do arquivo com o metodo .read()
    text = file.read()

# Pega os *ultimos* 2000 caracteres -- note o sinal de menos antes do numero
extract = text[-2000:]

# Imprime o resultado
print(extract)
```

O texto tem muitos erros de OCR, como seqüências e ,,,,. Compilamos uma expressão que busca **dois ou mais pontos finais** com `compile()`. O prefixo `r` guarda a *string* em formato “bruto” (raw):

Python

```
# Compila uma expressao regular e a atribui a variavel 'stops'
stops = re.compile(r'\.{2,}')
```

```
# Vamos verificar o tipo da expressao regular!
type(stops)
```

Saída

```
re.Pattern
```

- A **barra invertida** `\` antes do `.` diz que queremos o ponto “de verdade” (nas regex, `.` é um curinga).
- As **chaves** `{2,}` buscam o item anterior **duas ou mais vezes**.

Aplicamos com `sub()`, que recebe `repl` (o substituto) e `string` (onde procurar):

Python

```
# Aplica a expressao regular ao texto em 'extract' e salva a saida
# na mesma variavel, essencialmente sobrescrevendo o texto antigo.
extract = stops.sub(repl='', string=extract)

# Imprime o texto para examinar o resultado
print(extract)
```

As seqüências de pontos finais desapareceram.

5.2 Alternativas na expressão regular

Acrescentando **alternativas** com parênteses `()` e a barra vertical `|`, buscamos **dois ou mais pontos finais ou vírgulas**:

Python

```
# Compila uma expressao regular e a atribui a variavel 'punct'
punct = re.compile(r'(\.|\,){2,}')
```

Reiniciamos `extract` a partir de `text` e aplicamos o novo padrão:

Python

```
# "Reinicia" a variavel extract pegando os ultimos 2000 caracteres da string
original
extract = text[-2000:]

# Aplica a expressao regular
extract = punct.sub(repl='', string=extract)

# Imprime o resultado
print(extract)
```

Sucesso! Pontos finais e vírgulas foram removidos com uma única expressão. Sequências mais irregulares exigiriam padrões mais complexos.

Dica

Use um serviço como regex101.com para aprender e testar expressões regulares interativamente. E lembre-se: para identificar padrões confiáveis, observe sempre **mais de um** texto do seu corpus.

5.3 Processando múltiplos arquivos

Corpora costumam ter textos em vários arquivos. A classe `Path` do módulo `pathlib` simplifica abrir, ler, processar e fechar arquivos programaticamente — e **infere automaticamente** o tipo de caminho do sistema operacional (Windows, Linux, macOS):

Python

```
from pathlib import Path
```

Python

```
# Cria um objeto Path que aponta para o diretorio 'data' e o atribui
# a variavel 'corpus_dir'
corpus_dir = Path('data')
```

O objeto oferece métodos úteis que retornam valores **booleanos** (`True/False`):

Python

```
# Usa o metodo exists() para verificar se o caminho e valido
corpus_dir.exists()
```

Saída

True

Python

```
# Usa o metodo is_dir() para verificar se o caminho aponta para um diretorio
corpus_dir.is_dir()

# Usa o metodo is_file() para verificar se o caminho aponta para um arquivo
corpus_dir.is_file()
```

Saída

True
False

Usamos `glob()` (de *global*) para coletar os arquivos `.txt`. O `*` é um **curinga**; convertemos o resultado em lista com `list()`:

Python

```
# Coleta todos os arquivos com sufixo .txt no diretorio 'corpus_dir' e converte o
resultado em uma lista
files = list(corpus_dir.glob(pattern='*.txt'))

# Mostra o resultado
files
```

Saída

```
[PosixPath('data/WP_1990-08-10-25A.txt'),
 PosixPath('data/NYT_1991-01-16-A15.txt'),
 PosixPath('data/WP_1991-01-17-A1B.txt')]
```

Com a lista de três `Path`, iteramos e gravamos cada arquivo modificado com um novo nome:

Python

```
# Percorre a lista de objetos Path em 'files'. Refere-se a cada arquivo
# pela variavel 'file'.
for file in files:

    # Usa o metodo read_text() de um objeto Path para ler o conteudo do arquivo.
    # Passa o valor 'utf-8' ao argumento 'encoding' para declarar a codificacao.
    # Guarda o resultado na variavel 'text'.
    text = file.read_text(encoding='utf-8')

    # Aplica a expressao regular definida acima para remover a pontuacao
    # excessiva do texto. Guarda o resultado na variavel 'mod_text'.
    mod_text = punct.sub('', text)
```

```

# Define um novo nome de arquivo com o prefixo 'mod_' criando uma nova string.
# O objeto Path guarda o nome do arquivo como string no atributo 'name'.
# Combina as duas strings com o operador '+'.
new_filename = 'mod_' + file.name

# Define um novo objeto Path que aponta para o novo arquivo. O objeto Path
# junta automaticamente o diretório e o nome do arquivo para nós.
new_path = Path('data', new_filename)

# Imprime uma mensagem de status usando formatação de string. Ao adicionar
# o prefixo 'f' a uma string, podemos usar chaves {} para inserir uma
# variável dentro da string. Aqui inserimos o caminho atual do arquivo.
print(f'Writing modified text to {new_path}')

# Usa o método write_text() para escrever o texto modificado de 'mod_text'
# no arquivo, com codificação UTF-8.
new_path.write_text(mod_text, encoding='utf-8')

```

Saída

```

Writing modified text to data/mod_WP_1990-08-10-25A.txt
Writing modified text to data/mod_NYT_1991-01-16-A15.txt
Writing modified text to data/mod_WP_1991-01-17-A1B.txt

```

Os objetos `Path` oferecem `read_text()` e `write_text()`, que leem e gravam **sem** o `with` — fechando o arquivo automaticamente após a leitura.

Quiz

Marque a alternativa correta (a certa está marcada com **(correta)**).

1. Qual formato de texto é o mais interoperável para programação?

1. Texto formatado (*rich text*)
2. Texto estruturado (*structured text*)
3. Texto simples (*plain text*) **(correta)**

2. Num texto, a sequência de dois caracteres “barra-n” representa:

1. Um espaço em branco
2. Uma quebra de linha **(correta)**
3. Uma tabulação

3. O UTF-8 é retrocompatível com qual codificação?

1. ASCII **(correta)**
2. Latin-1
3. UTF-16

4. Qual método de string divide o texto em uma lista?

1. `join()`

2. `split()` (correta)

3. `replace()`

5. Numa expressão regular, o quantificador “dois ou mais” é escrito como:

1. `{2}`

2. `{2,}` (correta)

3. `{,2}`

6. Qual método da classe `Path` coleta arquivos por um padrão com curinga?

1. `glob()` (correta)

2. `read_text()`

3. `exists()`

Resumo da unidade

Nesta unidade, você aprendeu a

1. distinguir texto formatado, estruturado e simples;
2. compreender a codificação de caracteres (ASCII e Unicode/UTF-8);
3. carregar arquivos com `open()` + `with` + `read()`;
4. manipular *strings* com `replace()`, `split()`, `pop()` e `join()`;
5. definir padrões com expressões regulares (`re.compile()`, `sub()`);
6. processar vários arquivos com `pathlib.Path`, `glob()`, `read_text()` e `write_text()`.

Exercícios sugeridos

1. Carregue um arquivo do diretório `data` e conte quantas quebras de linha (`\n`) ele contém antes de qualquer limpeza.
2. Escreva um *pipeline* (lista de tuplas) que padronize travessões em um único caractere.
3. Crie uma expressão regular que substitua dois ou mais espaços em branco por um único espaço.
4. Adapte o laço da Seção 5.3 para gravar os arquivos modificados em um diretório `output`.

Referências. *Applied Language Technology* (MOOC), Universidade de Helsinque — applied-language-technology.mooc.fi. Hill, M. J.; Hengchen, S. (2019). *Quantifying the impact of dirty OCR on historical text analysis*. Digital Scholarship in the Humanities.

Material produzido para o Programa CiberExt 26-29 / FEELT38103 — Universidade Federal de Uberlândia.